

**UNIVERSIDAD AUTÓNOMA DE BAJA
CALIFORNIA**
FACULTAD DE CIENCIAS QUIMICAS E
INGENIERIA



Nombre de la materia

Ingeniería de software

Reporte final de Proyecto

Sistema de Gestión de Citas Médicas de Calendarización

Nombre del alumno(s), matrícula(s):

José Raziel Trujeque Castor – 1276448

Tepale Chocolatel Armando - 1280222

Andrea Sofia Chavez Munguia - 1279990

Tijuana Baja California, Baja California a 29 de mayo del 2025.

Instalaciones	5
1. Introducción.	6
1.1 Generalidades de la asignatura.	6
Propósito de la asignatura:	6
Competencia general:	6
Evidencias de desempeño:	7
1.2 Metodología de desarrollo a utilizar para implementar el proyecto.	7
Aplicación de Scrum en el proyecto.	7
Asignación de roles en el equipo Scrum.	8
Ventajas observadas con el uso de Scrum.	8
2. Principales funcionalidades del sistema.	8
2.1 Gestión de citas médicas.	8
2.3 Reportes e indicadores administrativos.	9
2.4 Gestión de usuarios y control de accesos.	9
2.5 Portal web para pacientes.	9
2.6 Seguridad y protección de datos.	9
3. Perfil de los usuarios.	9
3.1 Usuarios por funcionalidad.	10
3.2 Perfil de usuario objetivo.	11
Administrativo.	11
Médico.	11
Paciente / Tutor.	11
Coordinación de informática.	11
4. Requerimientos del sistema.	12
4.1. Requerimientos funcionales (“F” del modelo FURPS).	12
Agendamiento de citas.	12
Disponibilidad y Rendimiento	13
Gestión de Citas	13
Compatibilidad y Usabilidad	14
Control de Disponibilidad	14
Control de Disponibilidad (Bloqueos y Actualizaciones en Tiempo Real)	14
Carga y Gestión de Horarios	15
Políticas y Restricciones	15
4.2. Requerimientos no funcionales (“URPS” del modelo FURPS).	16
Seguridad y Privacidad	16
Disponibilidad y Rendimiento	16
Arquitectura y Mantenibilidad	16
Escalabilidad y Adaptabilidad	17
Compatibilidad y Consistencia en la Experiencia de Usuario	17
4.3. Priorización de los requerimientos	18
Must have (M) – Requerimientos que el sistema debe incluir obligatoriamente:	18

Should have (S) – Alta prioridad, se deberían incluir si es posible:	18
Could have (C) – Deseables, se podrían incluir si no representan mucho costo o esfuerzo:	18
Won't have (W) – No se incluirán en esta entrega, pero se consideran para el futuro:	18
5. Descripción de las herramientas utilizadas.	19
5.1. Lenguajes de programación.	19
5.2. Entornos/Frameworks de desarrollo.	19
5.3. Herramientas de diseño.	19
6. El Product Backlog .	20
6.1. Traducir los requerimientos FURPS a USER STORIES.	20
6.2. El Product Backlog	21
6.3. Calendarización del Product Backlog en Sprints	23
7. Análisis de factibilidad del proyecto.	23
7.1 Factibilidad técnica.	23
a) Funcionalidades del sistema.	23
b) Tecnologías utilizadas.	24
Evaluación de la factibilidad técnica	25
7.2 Factibilidad operativa.	25
a) Recurso humano disponible y capacidades	25
b) Roles en la metodología Scrum	26
c) Plan de comunicación con el cliente y usuarios	26
7.3 Análisis FODA	27
Fortalezas (F)	27
Oportunidades (O)	27
Debilidades (D)	27
Amenazas (A)	28
8.0 Análisis de Riesgos	28
8.1. Identificación de los riesgos	28
8.2. Caracterización de los riesgos	29
8.3. Acciones para enfrentar los riesgos	30
Prevención:	30
Recuperación:	30
9. Desarrollo por Sprints.	30
9.1. Sprint 1	30
a) Sprint Planning	30
b) Descripción de User Stories (Sprint 1)	31
c) Desarrollo de User Stories	33
d) Pruebas de funcionalidades	34
9.2. Sprint 2	36
a) Sprint Planning	36
b) Descripción de User Stories (Sprint 2)	36

c) Desarrollo de User Stories	37
d) Pruebas de funcionalidades	38
9.3. Sprint 3	40
a) Sprint Planning	40
b) Descripción de User Stories (Sprint 3)	40
c) Desarrollo de User Stories	41
d) Pruebas de funcionalidades	42
10. Estimación del costo del proyecto	43
10.1. Estimación basada en Story Points y Sprints.	44
10.2. Distribución de horas por rol	44
10.3. Estimación total del proyecto.	45
10.4. Conclusión de la estimación.	45
11. Conclusiones.	45
12. Referencias.	47
13. Apéndices.	50
Comentarios finales	56

Instalaciones



1. Introducción.

El presente documento expone el desarrollo y resultados del proyecto integrador realizado como parte de la asignatura Ingeniería de Software, impartida en la Facultad de Ingeniería de la Universidad Autónoma de Baja California (UABC). Este proyecto consistió en la planeación, diseño, desarrollo y documentación de un Sistema de Gestión de Citas Médicas de Calendarización, realizado en colaboración con el Hospital Infantil de las Californias, con el objetivo de automatizar procesos administrativos y mejorar la atención clínica a los pacientes.

A lo largo del semestre, los estudiantes participaron en un entorno de simulación profesional donde aplicaron metodologías modernas de desarrollo de software, modelado UML, análisis de requerimientos, estimación de costos, aseguramiento de calidad y documentación técnica. Esta experiencia no solo reforzó los conocimientos teóricos, sino también habilidades prácticas necesarias para el ejercicio profesional en el ámbito de las tecnologías de la información.

El desarrollo del sistema fue guiado por los lineamientos de la asignatura, combinando el aprendizaje técnico con el trabajo en equipo, la retroalimentación continua y la interacción directa con un cliente real, permitiendo así la aplicación integral de todos los conceptos vistos durante el curso.

1.1 Generalidades de la asignatura.

La asignatura de Ingeniería de Software tiene como objetivo principal formar estudiantes con la capacidad de analizar, diseñar, construir y mantener sistemas de software de calidad, utilizando enfoques estructurados y metodologías ágiles, de acuerdo con estándares profesionales. Esta materia forma parte del núcleo formativo de la carrera de Ingeniería en Computación y es fundamental para el desarrollo de soluciones informáticas en contextos reales.

Propósito de la asignatura:

- Introducir a los estudiantes en el proceso sistemático del desarrollo de software, desde la especificación de requerimientos hasta la entrega del producto final.
- Desarrollar la capacidad para aplicar modelos y metodologías que garanticen la calidad del producto de software, como el modelo FURPS (Funcionalidad, Usabilidad, Fiabilidad, Desempeño y Soporte).

Competencia general:

- Diseñar y desarrollar sistemas de software aplicando metodologías de ingeniería que aseguren la calidad, eficiencia y adecuación del producto a las necesidades del usuario, mediante el uso de modelos de procesos, técnicas de modelado, pruebas, y buenas prácticas de documentación y mantenimiento.

Evidencias de desempeño:

Durante el semestre, se generaron las siguientes evidencias:

- Análisis de requerimientos y redacción de historias de usuario.
- Diseño de diagramas UML (casos de uso, actividades, clases y secuencia).
- Documentación técnica del proyecto.
- Aplicación de metodologías ágiles (Scrum) para la gestión del proyecto.
- Desarrollo de un sistema web funcional, probado y documentado.
- Presentación final del sistema a un cliente real.

1.2 Metodología de desarrollo a utilizar para implementar el proyecto.

El desarrollo del proyecto se llevó a cabo aplicando la metodología ágil Scrum, una de las más utilizadas en la industria del software debido a su flexibilidad, enfoque iterativo e incremental, y orientación a la colaboración entre el equipo de desarrollo y el cliente. Esta metodología permite entregar valor de forma continua mediante la división del proyecto en ciclos cortos de trabajo llamados Sprints, que en este caso fueron de dos semanas.

Aplicación de Scrum en el proyecto.

El marco de trabajo Scrum fue implementado de la siguiente manera:

- **Sprint Planning (Planificación del Sprint):** Al inicio de cada Sprint, se definieron los objetivos específicos y se seleccionaron las historias de usuario prioritarias del Product Backlog que serían desarrolladas en la iteración.
- **Daily Scrum (Reuniones Diarias):** Se realizaron reuniones breves en las que cada integrante compartía su avance, obstáculos encontrados y sus tareas siguientes. Esto fomentó la comunicación y ayudó a detectar problemas a tiempo.
- **Sprint Review (Revisión del Sprint):** Al final de cada Sprint se realizó una demostración del producto al cliente (Hospital Infantil de las Californias), quien proporcionó retroalimentación directa sobre el cumplimiento de los requerimientos.
- **Sprint Retrospective (Retrospectiva del Sprint):** El equipo reflexionó sobre lo que funcionó bien, lo que se podía mejorar y estableció acciones concretas para optimizar su desempeño en el siguiente Sprint.

Asignación de roles en el equipo Scrum.

- **Scrum Master:** Raziel Trujeque
Facilitador del proceso Scrum. Se encargó de remover impedimentos y garantizar el cumplimiento de la metodología.
- **Product Owner:** Armando Tepale
Representó al cliente y priorizó el Backlog, asegurando que las funcionalidades desarrolladas se alinearan con los objetivos del hospital.
- **Equipo de Desarrollo:** Carlos Vela y Sofia Chavez
Carlos se enfocó en el análisis, pruebas e integración del backend; mientras que Sofia trabajó en el diseño, lógica de interfaz y experiencia de usuario.

Ventajas observadas con el uso de Scrum.

- **Adaptabilidad:** Se logró responder rápidamente a cambios solicitados por el hospital sin afectar negativamente el progreso general del proyecto.
- **Entregas funcionales frecuentes:** Al finalizar cada Sprint se entregaron versiones utilizables del sistema, lo que fortaleció la confianza del cliente.
- **Mejora continua:** Las retrospectivas permitieron ajustar la forma de trabajo en cada iteración, reduciendo errores y mejorando la eficiencia.
- **Colaboración constante:** El enfoque participativo de Scrum fortaleció el trabajo en equipo y promovió la toma de decisiones compartida.

2. Principales funcionalidades del sistema.

El sistema de Gestión De Citas Médicas de Calendarización desarrollado para el Hospital Infantil de las Californias fue diseñado con el objetivo de optimizar procesos administrativos y clínicos a través de una plataforma interactiva, modular, con componentes gráficos amigables, interfaz responsiva y montada en un web hosting funcional, asegurando su accesibilidad desde cualquier dispositivo con conexión a internet.

El desarrollo partió de requerimientos específicos obtenidos mediante entrevistas con personal del hospital, lo que permitió crear funcionalidades claras, bien definidas y alineadas con las necesidades reales. A continuación, se listan y describen las principales funciones de la aplicación:

2.1 Gestión de citas médicas.

Permite programar, visualizar y gestionar citas fácilmente por parte del personal y los pacientes:

- **Creación y edición de citas:** Selección de especialidad, médico y horario.
- **Agenda médica interactiva:** Vista tipo calendario (día, semana, mes) con componentes gráficos dinámicos.

- **Notificaciones automáticas:** Envío de correos electrónicos para recordar, cancelar o confirmar citas.
- **Registro de asistencia:** Control de pacientes que asistieron o no a sus citas.

2.3 Reportes e indicadores administrativos.

Herramienta para la toma de decisiones basada en datos reales:

- **Consultas por médico:** Reporte del número de pacientes atendidos por profesional.
- **Flujo de pacientes:** Visualización por hora o día para organizar recursos.

2.4 Gestión de usuarios y control de accesos.

Garantiza la seguridad y confidencialidad del sistema mediante una jerarquía de roles:

- **Perfiles personalizados:** Acceso por tipo de usuario (médico, paciente, administrador, etc.).
- **Permisos por módulo:** Restricción de funcionalidades según el rol asignado.
- **Bitácora de actividades:** Registro de cambios y accesos para auditorías.

2.5 Portal web para pacientes.

Una interfaz moderna, responsiva y centrada en mejorar la experiencia del usuario externo:

- **Historial de salud accesible:** Visualización de citas previas, tratamientos y diagnósticos.
- **Agendamiento de citas:** Solicitud y modificación de citas sin intervención del personal.

2.6 Seguridad y protección de datos.

Funcionalidades diseñadas para cumplir con normativas de privacidad y ciberseguridad:

- **Cifrado de contraseñas:** Mediante algoritmos hash.
- **Protección contra ataques:** Medidas frente a inyecciones SQL y XSS.
- **Control de sesiones:** Expiración automática y cierre de sesión seguro.
- **RespalDOS periódicos:** Automatización de copias de seguridad de la base de datos.

3. Perfil de los usuarios.

El sistema de Gestión de Citas Médicas fue diseñado para atender a diferentes tipos de usuarios dentro del Hospital Infantil de las Californias, considerando tanto sus funciones operativas como su nivel de experiencia tecnológica. A continuación, se detallan los miembros del equipo de desarrollo Scrum, los usuarios asignados a cada funcionalidad del sistema, y el perfil objetivo para cada tipo de usuario final.

Asignación de roles en el equipo Scrum.

- **Scrum Master: Raziell Trujeque.**
Facilitador del proceso Scrum. Se encargó de eliminar impedimentos durante el desarrollo y garantizar que se cumplieran los principios y prácticas ágiles del marco de trabajo Scrum.
- **Product Owner: Armando Tepale.**
Representó los intereses del cliente (el hospital), priorizó el Product Backlog y validó que las funcionalidades desarrolladas estuvieran alineadas con las necesidades reales de la institución.
- **Equipo de Desarrollo: Carlos Vela y Sofia Chavez.**
 - **Carlos Vela** se encargó del análisis de requerimientos, realización de pruebas de calidad (QA) e integración del backend con el frontend.
 - **Sofia Chavez** desarrolló el diseño visual, lógica de interfaz y trabajó en la experiencia de usuario, garantizando que la aplicación fuera clara, funcional y responsiva.

3.1 Usuarios por funcionalidad.

Funcionalidad.	Usuario principal.
Alta de doctores.	Recepcionista (Administrador).
Programación de citas.	Recepcionista, Tutor, Administrador.
Reportes administrativos.	Administrador.
Alta de especialidades y servicios.	Administrador.
Gestión de usuarios y accesos.	Sistema (Vista Administrador).
Solicitud de citas en línea.	Tutores (Usuarios).

3.2 Perfil de usuario objetivo.

El sistema contempla distintos perfiles de usuarios, cada uno con características demográficas y tecnológicas particulares:

Administrativo.

- **Edad promedio:** 25 a 55 años
- **Grado de estudios:** Bachillerato o licenciatura en áreas administrativas
- **Conocimientos:** Uso básico de computadora (correo electrónico, navegación web, Office)
- **Coeficiente tecnológico:** Medio. Puede adaptarse con capacitación breve al uso del sistema.

Médico.

- **Edad promedio:** 28 a 60 años
- **Grado de estudios:** Licenciatura o especialidad médica
- **Conocimientos:** Nivel intermedio en tecnología. Acostumbrado al uso de software de expedientes o sistemas hospitalarios.
- **Coeficiente tecnológico:** Medio-Alto. Requiere eficiencia y velocidad de acceso, sobre todo desde consultorios.

Paciente / Tutor.

- **Edad promedio:** 25 a 50 años (tutores)
- **Grado de estudios:** Diverso (secundaria a licenciatura)
- **Conocimientos:** Uso general de smartphones o navegación web
- **Coeficiente tecnológico:** Bajo-Medio. Por eso se priorizó una interfaz web sencilla, clara y accesible.

Coordinación de informática.

- **Edad promedio:** 20 a 45 años
- **Grado de estudios:** Ingeniería en sistemas o carrera técnica en informática
- **Conocimientos:** Avanzados en servidores, bases de datos y mantenimiento de software
- **Coeficiente tecnológico:** Alto. Responsable del hosting, respaldos, seguridad y soporte general.

4. Requerimientos del sistema.

Los requerimientos del sistema se definieron con base en el modelo FURPS (Funcionalidad, Usabilidad, Confiabilidad, Rendimiento y Soporte), el cual permitió estructurar de manera clara tanto las necesidades funcionales como las no funcionales del proyecto. Esta clasificación facilitó la validación con el Product Owner y la priorización durante los sprints de desarrollo.

El sistema fue diseñado para cumplir con los estándares de calidad requeridos por el Hospital Infantil de las Californias, asegurando una experiencia adecuada para usuarios administrativos, médicos, pacientes y personal técnico.

4.1. Requerimientos funcionales (“F” del modelo FURPS).

Agendamiento de citas.

RF1 – Verificación de Datos del Paciente:

“El usuario TUTOR debe poder agendar una cita para un paciente, con base al número de expediente del paciente. La cita se agenda con un médico específico, en un día y hora deseable por el Tutor y con base a la disponibilidad de citas. Se debe tener como resultado la CITA AGENDADA.”

RF2 – Mostrar Médicos Disponibles:

“El usuario TUTOR debe poder visualizar la lista de médicos disponibles según la especialidad seleccionada, ya que el sistema consulta en tiempo real la base de datos y muestra la disponibilidad actual de cada profesional.”

RF2.1 – Selección de Médico:

“El usuario TUTOR debe poder seleccionar el médico deseado de la lista presentada, apreciando detalles relevantes como experiencia, especialización y disponibilidad, con confirmación mediante notificaciones.”

RF3 – Selección de Fecha:

“El usuario TUTOR debe poder elegir la fecha de la cita mediante un calendario o selector de fechas, en el cual el sistema valida que la fecha sea correcta y esté dentro del rango permitido.”

RF4 – Selección de Hora:

“El usuario TUTOR debe poder seleccionar la hora de la cita en función de la disponibilidad del médico para la fecha elegida, a través de un listado actualizado en tiempo real que evite solapamientos.”

Disponibilidad y Rendimiento

RF5 – Agendamiento para el Mismo Día:

“El usuario TUTOR debe poder agendar una cita para el mismo día si existe disponibilidad inmediata de médicos y horarios, ya que el sistema actualiza la agenda en tiempo real y valida la viabilidad del agendamiento.”

RF5.1 – Mensaje de No Disponibilidad:

“Si no hay horarios o médicos disponibles para el mismo día, el sistema mostrará un mensaje claro al tutor, informando sobre la falta de disponibilidad para ese día. De ser posible, el sistema sugerirá fechas alternativas, facilitando la reprogramación de la cita.”

Gestión de Citas

RF6 – Visualización de Citas Agendadas:

“El usuario TUTOR debe poder consultar el listado de citas agendadas para su paciente, donde el sistema actualiza en tiempo real la información para evitar duplicidades y facilitar la gestión.”

RF7 – Visualización del Estado de la Cita:

“El usuario TUTOR debe poder conocer el estado actual de una cita previamente agendada, observando si está pendiente, aprobada, rechazada o cancelada, gracias a las actualizaciones automáticas del sistema.”

RF8 – Visualización de Datos Generales de la Cita:

“El usuario TUTOR debe poder acceder a todos los datos generales de una cita, como fecha, hora, médico asignado y ubicación, para confirmar y gestionar los detalles de la misma.”

RF9 – Visualización de Datos del Paciente:

“El usuario TUTOR debe poder visualizar en tiempo real la información completa de la paciente asociada a una cita, incluyendo datos personales y clínicos. La interfaz mostrará estos datos de forma organizada y fácil de comprender, asegurando el cumplimiento de las normativas de privacidad y seguridad para proteger la información sensible.”

RF10 – Cancelación de Citas:

“El usuario TUTOR debe poder cancelar una cita previamente agendada a través de una opción clara y accesible en el sistema. Al seleccionar la opción de cancelación, se solicitará una confirmación mediante un diálogo interactivo para evitar cancelaciones accidentales. Una vez confirmada, el sistema enviará notificaciones automáticas (por ejemplo, vía correo electrónico) a todas las partes involucradas y registrará el evento de cancelación para futuros análisis o para la aplicación de restricciones.”

Compatibilidad y Usabilidad

RF11 – Recordatorios de Citas:

“El usuario TUTOR debe recibir recordatorios automáticos para sus citas, ya sea a través de correo electrónico o notificaciones internas. Estos recordatorios se enviarán en intervalos predefinidos (un día y una hora antes de la cita), ayudando a minimizar la inasistencia y garantizando que esté siempre informado sobre sus próximas citas.”

Control de Disponibilidad

RF12 – Historial de Citas:

“El usuario TUTOR debe poder consultar un historial completo de todas las citas (realizadas, pendientes o canceladas) de manera ordenada y segura. La interfaz permitirá revisar eventos pasados y analizar tendencias, facilitando la gestión de la información.”

RF13 – Generación de Reportes:

“El usuario ADMINISTRADOR debe poder generar reportes detallados sobre las citas agendadas, aplicando filtros por médico, especialidad y período de tiempo. Los resultados se presentarán en formatos visuales, como gráficos y tablas, que faciliten la toma de decisiones y el análisis del desempeño.”

RF14 – Modificación de Citas:

“El usuario ADMINISTRADOR debe poder modificar una cita previamente agendada, actualizando detalles como fecha, hora y médico asignado. Al realizar estos cambios, el sistema actualizará la información en tiempo real y enviará notificaciones automáticas a las partes involucradas, garantizando que todos los usuarios dispongan de los datos correctos y actualizados.”

Control de Disponibilidad (Bloqueos y Actualizaciones en Tiempo Real)

RF15.1 – Bloqueo Inmediato de Horario:

“El usuario TUTOR debe poder confirmar la selección de un horario, el cual se bloqueará de forma inmediata en el sistema para evitar que otros usuarios lo reserven simultáneamente. Este bloqueo se aplicará en tiempo real, asegurando la integridad de la agenda y previniendo conflictos en el agendamiento.”

RF15.2 – Actualización Instantánea de Disponibilidad:

“El usuario TUTOR debe poder observar que, en el momento en que se bloquea un horario, el sistema actualiza instantáneamente la disponibilidad en todas las interfaces. Esto garantiza que solo se muestren opciones disponibles y se eviten conflictos en el agendamiento.”

RF15.3 – Liberación de Horario Bloqueado:

“El usuario TUTOR debe poder iniciar el proceso de agendamiento con la

seguridad de que, si no se confirma en un plazo determinado, el sistema liberará automáticamente el horario bloqueado, permitiendo que otros usuarios puedan reservar ese espacio.”

RF15.4 – Actualización de Vista de Horarios:

“El usuario TUTOR debe ver actualizada en tiempo real la vista de horarios disponibles. Si otro usuario reserva un horario mientras se consulta la disponibilidad, el sistema actualizará inmediatamente la interfaz para reflejar la nueva situación y evitar mostrar opciones ya reservadas.”

Carga y Gestión de Horarios

RF16 – Carga de Horarios Predefinidos:

“El usuario ADMINISTRADOR debe poder cargar y actualizar de forma masiva los horarios disponibles de cada médico según su disponibilidad, facilitando tanto la configuración inicial de la agenda como la gestión dinámica de citas ante cambios en la disponibilidad de los profesionales.”

Políticas y Restricciones

RF17.1 – Impedir Agendamiento Inmediato:

“El usuario TUTOR debe poder observar que, si un paciente ha cancelado una cita en las últimas 24 horas, el sistema impedirá el agendamiento inmediato de una nueva cita. Esta validación se realizará en tiempo real, asegurando que el historial de cancelaciones se verifique y se evite el uso inadecuado o abusivo del servicio.”

RF17.2 – Mensaje de Error (Cancelación Reciente):

“El usuario TUTOR debe recibir un mensaje de error claro y descriptivo si intenta agendar una nueva cita para un paciente que canceló recientemente (dentro de las últimas 24 horas). El mensaje indicará la restricción y brindará instrucciones o sugerencias sobre cuándo se podrá reprogramar la cita.”

RF17.3 – Registro de Cancelación:

“El usuario ADMINISTRADOR debe contar con un registro seguro y detallado de la fecha y hora de cada cancelación de cita. Este registro se utilizará para validar futuras restricciones en el agendamiento y para realizar auditorías y análisis estadísticos sobre el comportamiento de las cancelaciones.”

RF18 – Configuración de Políticas de Agendamiento:

“El usuario ADMINISTRADOR debe poder configurar las políticas internas de agendamiento a través de una interfaz intuitiva, estableciendo parámetros como la cantidad máxima de citas permitidas por paciente en

un mismo día, para adaptarse a distintos escenarios operativos y optimizar el uso de los recursos.”

4.2. Requerimientos no funcionales (“URPS” del modelo FURPS).

Seguridad y Privacidad

RNF1 – Protección de Información del Paciente:

El sistema implementará un control de acceso basado en roles, de modo que sólo los usuarios autorizados (como tutores y administradores) puedan acceder a la información sensible del paciente. Se utilizarán métodos de autenticación robustos y se asignan permisos específicos, garantizando la confidencialidad y confiabilidad de los datos.

RNF2 – Privacidad de Datos:

Tanto el diseño como la operación del sistema cumplirán con todas las normativas vigentes en materia de privacidad de datos, tales como HIPAA y GDPR. Se adoptarán medidas técnicas y organizativas para asegurar la protección y confidencialidad de la información personal y clínica.

RNF3 – Mecanismos de Seguridad Adicionales: Se implementarán medidas de seguridad adicionales, como la expiración automática de sesiones inactivas, para prevenir accesos no autorizados en caso de que un usuario deje su sesión abierta, reduciendo así el riesgo de vulneraciones.

Disponibilidad y Rendimiento

RNF4 – Disponibilidad 24/7:

El sistema garantizará un funcionamiento continuo, estando disponible las 24 horas del día y los 7 días de la semana. Para ello se implementarán soluciones de redundancia, respaldo y recuperación ante fallos, minimizando el tiempo de inactividad y asegurando altos niveles de confiabilidad y usabilidad.

RNF5 – Optimización de Carga para Redes de Baja Velocidad:

Con el fin de garantizar una experiencia de usuario aceptable en entornos con conectividad limitada, el sistema optimizará la carga de recursos (imágenes, scripts, hojas de estilo, etc.) mediante técnicas de compresión y carga progresiva, reduciendo los tiempos de espera y mejorando el rendimiento.

Arquitectura y Mantenibilidad

RNF6 – Diseño Modular para Mantenibilidad:

El sistema se estructurará en módulos independientes y bien definidos, lo que facilitará la realización de actualizaciones, correcciones y expansiones sin impactar el funcionamiento global. Esta modularidad es clave para la escalabilidad y el mantenimiento a largo plazo.

Escalabilidad y Adaptabilidad

RNF7 – Escalabilidad del Sistema:

El sistema estará diseñado para soportar un incremento continuo en el número de usuarios y en la cantidad de citas gestionadas. Se logrará mediante una arquitectura que permita la expansión de recursos, tanto de forma horizontal como vertical, sin comprometer el rendimiento.

RNF8 – Adaptación a Cambios Futuros:

El diseño del sistema será lo suficientemente flexible como para incorporar nuevas funcionalidades o adaptarse a cambios en los requerimientos de negocio y normativas. Una arquitectura modular y escalable facilitará la integración de nuevas tecnologías y procesos sin necesidad de una reestructuración completa.

RNF9 – Escalabilidad Vertical y Horizontal:

La arquitectura permitirá aumentar la capacidad de procesamiento de servidores individuales (escalabilidad vertical) y distribuir la carga entre múltiples servidores o instancias (escalabilidad horizontal), asegurando que el sistema pueda manejar picos de demanda sin degradar su rendimiento.

Compatibilidad y Consistencia en la Experiencia de Usuario

RNF10 – Compatibilidad con Navegadores Web:

El desarrollo del sistema se basará en estándares web que aseguren su correcto funcionamiento en los navegadores más utilizados (Chrome, Firefox, Safari, Edge, etc.). La interfaz se adaptará a las particularidades de cada navegador, garantizando una experiencia uniforme y de alta calidad.

RNF11 – Consistencia en la Experiencia del Usuario:

El diseño de la interfaz será responsivo y adaptativo, asegurando que la aplicación ofrezca una experiencia coherente y de alta calidad en distintos dispositivos (PC, tablet, móvil) y resoluciones de pantalla.

RNF12 – Funcionamiento Óptimo en Diversos Dispositivos:

El sistema estará optimizado para funcionar de manera óptima en una amplia variedad de dispositivos y tamaños de pantalla, implementando un diseño adaptativo que mantiene la funcionalidad completa y una presentación visual adecuada en cada entorno.

4.3. Priorización de los requerimientos

Must have (M) – Requerimientos que el sistema debe incluir obligatoriamente:

- **RF1** – Verificación de Datos del Paciente.
- **RF2** – Mostrar Médicos Disponibles.
- **RF3** – Selección de Fecha.
- **RF4** – Selección de Hora.
- **RF6** – Visualización de Citas Agendadas.
- **RF7** – Visualización del Estado de la Cita.
- **RF8** – Visualización de Datos Generales de la Cita.
- **RF9** – Visualización de Datos del Paciente.
- **RF10** – Cancelación de Citas.
- **RF15.1** – Bloqueo Inmediato de Horario.
- **RF15.2** – Actualización Instantánea de Disponibilidad.
- **RNF1** – Protección de Información del Paciente.
- **RNF2** – Privacidad de Datos.
- **RNF4** – Disponibilidad 24/7.
- **RNF10** – Compatibilidad con Navegadores Web.
- **RNF11** – Consistencia en la Experiencia del Usuario.
- **RNF12** – Funcionamiento Óptimo en Diversos Dispositivos.

Should have (S) – Alta prioridad, se deberían incluir si es posible:

- **RF2.1** – Selección de Médico.
- **RF5** – Agendamiento para el Mismo Día.
- **RF12** – Historial de Citas.
- **RF13** – Generación de Reportes.
- **RF14** – Modificación de Citas.
- **RNF3** – Mecanismos de Seguridad Adicionales.
- **RNF5** – Optimización de Carga para Redes de Baja Velocidad.
- **RNF6** – Diseño Modular para Mantenibilidad.
- **RNF7** – Escalabilidad del Sistema.
- **RNF8** – Adaptación a Cambios Futuros.

Could have (C) – Deseables, se podrían incluir si no representan mucho costo o esfuerzo:

- **RF5.1** – Mensaje de No Disponibilidad.
- **RF11** – Recordatorios de Citas.
- **RNF9** – Escalabilidad Vertical y Horizontal.

Won't have (W) – No se incluirán en esta entrega, pero se consideran para el futuro:

- **Ninguno especificado explícitamente**, aunque se pueden mover algunos "Could have" aquí si el tiempo o los recursos lo requieren.

5. Descripción de las herramientas utilizadas.

5.1. Lenguajes de programación.

Para el desarrollo del sistema se utilizaron los siguientes lenguajes:

- **PHP**: Lenguaje de programación del lado del servidor, encargado de la lógica de negocio y la conexión con la base de datos MySQL.
- **HTML5**: Utilizado para estructurar el contenido de las vistas del sistema.
- **CSS3**: Aplicado para el diseño visual.
- **JavaScript**: Empleado en la validación de formularios y elementos interactivos del lado del cliente, mejorando la experiencia del usuario.

5.2. Entornos/Frameworks de desarrollo.

Durante el desarrollo se integraron diversas herramientas y librerías que facilitaron la implementación de funcionalidades específicas:

- **Visual Studio Code**: Editor principal utilizado para escribir y organizar el código del proyecto.
- **FullCalendar JS**: Librería JavaScript empleada para la visualización dinámica de calendarios y la gestión interactiva de citas.
- **PHPMailer**: Librería utilizada para el envío automatizado de correos electrónicos con notificaciones sobre citas.
- **Trabajos de Cron**: Utilizados para programar tareas automáticas, como el envío de recordatorios o validaciones periódicas en segundo plano.

5.3. Herramientas de diseño.

Se emplearon distintas herramientas de apoyo tanto para el diseño de interfaz como para la organización del trabajo en equipo:

- **Visual Studio**: Utilizado para prototipado de algunas funcionalidades o pruebas de compatibilidad.
- **Notepad++**: Herramienta auxiliar para edición rápida de archivos de texto o código ligero.
- **Jira**: Plataforma utilizada para la gestión de tareas y seguimiento del avance del proyecto bajo metodología Scrum.
- **Notion**: Herramienta colaborativa utilizada para documentar requerimientos, reuniones, acuerdos y tableros del proyecto.
- **Discord**: Canal principal de comunicación en tiempo real del equipo, facilitando la coordinación entre los integrantes del desarrollo.
- **Herramienta para desarrolladores de navegadores**: Herramienta utilizada, para encontrar errores al momento de debugear la página web.

6. El Product Backlog .

6.1. Traducir los requerimientos FURPS a USER STORIES.

A continuación, se presentan las historias de usuario (US) derivadas de los requerimientos funcionales y no funcionales:

ID	Historia de Usuario (User Story)
US01	Como TUTOR, quiero agendar una cita con base en el número de expediente del paciente.
US02	Como TUTOR, quiero visualizar médicos disponibles según especialidad.
US03	Como TUTOR, quiero seleccionar un médico.
US04	Como TUTOR, quiero seleccionar una fecha de cita válida mediante un calendario.
US05	Como TUTOR, quiero seleccionar una hora disponible para la cita.
US06	Como TUTOR, quiero agendar una cita para el mismo día si hay disponibilidad.
US07	Como TUTOR, quiero recibir un mensaje si no hay disponibilidad de cita.
US08	Como TUTOR, quiero consultar citas ya agendadas.
US09	Como ADMINISTRADOR, quiero ver el estado de una cita (Disponible y Ocupado.).
US10	Como TUTOR, quiero consultar datos generales de la cita agendada.
US11	Como TUTOR, quiero ver los datos del paciente asociado a la cita.
US12	Como TUTOR, quiero cancelar una cita y recibir confirmación del sistema.
US13	Como TUTOR, quiero recibir recordatorios automáticos antes de mis citas.
US14	Como ADMIN, quiero generar reportes filtrando por médico, especialidad y fechas.
US15	Como ADMIN, quiero modificar una cita actualizando fecha, hora o médico.

US16	Como TUTOR, quiero que al confirmar un horario este se bloquee de inmediato.
US17	Como TUTOR, quiero ver actualizada en tiempo real la disponibilidad de horarios.
US18	Como ADMIN, quiero cargar masivamente los horarios disponibles de médicos.
US19	Como ADMIN, quiero configurar políticas de agendamiento (máx. por día, etc.).
US20	Como SISTEMA, quiero proteger datos sensibles mediante control de roles.
US21	Como SISTEMA, quiero cumplir con normativas como HIPAA y GDPR.
US22	Como SISTEMA, quiero cerrar automáticamente sesiones inactivas.
US23	Como SISTEMA, quiero estar disponible 24/7 con respaldo en caso de fallos.
US24	Como SISTEMA, quiero optimizar carga para redes lentas.
US25	Como SISTEMA, quiero estar modularmente construido para facilitar el mantenimiento.
US26	Como SISTEMA, quiero escalar para soportar más usuarios o funciones.
US27	Como SISTEMA, quiero funcionar correctamente en navegadores comunes.
US28	Como SISTEMA, quiero ser responsivo y adaptarse a móviles, tablets y PCs.

6.2. El Product Backlog

ID	Historia de Usuario (User Story)	Prioridad (MoSCoW)	Estado
US01	Agendar cita por expediente.	Must have (M)	Implementado
US02	Ver médicos por especialidad	Must have (M)	Implementado

US03	Seleccionar médico con detalles	Should have (S)	Implementado
US04	Elegir fecha con calendario	Should have (S)	implementado
US05	Seleccionar hora disponible	Must have (M)	Implementado
US06	Agendar cita el mismo día	Could have (C)	Implementado
US07	Mensaje si no hay disponibilidad	Should have (S)	Implementado
US08	Ver citas agendadas	Must have (M)	Implementado
US09	Ver estado de cita	Should have (S)	Implementado
US10	Ver datos de cita	Should have (S)	Implementado
US11	Ver datos del paciente	Must have (M)	Implementado
US12	Cancelar cita con confirmación	Must have (M)	Implementado
US13	Recordatorios de cita	Could have (C)	Implementado
US14	Generar reportes administrativos	Generar reportes administrativos	Futuro
US15	Modificar cita	Could have (C)	Implementado
US16	Bloqueo inmediato de horario	Must have (M)	Implementado
US17	Actualización en tiempo real	Must have (M)	Implementado
US18	Liberación automática de horario	Should have (S)	Futuro
US19	Configurar políticas de agendamiento	Won't have (W)	No se implementará
US20	Protección de información	Must have (M)	Implementado
US21	Cumplir normativas de privacidad	Should have (S)	Futuro
US22	Expirar sesión inactiva	Should have (S)	Parcialmente
US23	Disponibilidad 24/7	Must have (M)	Implementado
US24	Optimización para redes lenta	Could have (C)	Futuro
US25	Diseño modular	Must have (M)	Implementado
US26	Escalabilidad del sistema	Could have (C)	Futuro
US27	Compatibilidad con navegadores	Should have (S)	Implementado
US28	Experiencia responsiva	Must have (M)	Implementado

6.3. Calendarización del Product Backlog en Sprints

Sprint	User Stories incluidas
Sprint 1	US01, US02, US04, US05, US08, US11, US12
Sprint 2	US03, US07, US09, US10, US17, US18, US25, US30, US33
Sprint 3	US06, US13, US14, US16, US19, US26, US27, US29, US32
Futuro	US15, US20, US21, US22, US23, US24, US28

7. Análisis de factibilidad del proyecto.

7.1 Factibilidad técnica.

Desde el punto de vista técnico, el desarrollo del sistema de Gestión de Citas Médicas de Calendarización para el Hospital Infantil de las Californias fue altamente viable. La planificación y ejecución del proyecto se apoyaron en una correcta elección de tecnologías, una delimitación precisa de las funcionalidades y una distribución realista del alcance de desarrollo dentro del tiempo disponible del curso.

a) Funcionalidades del sistema.

El sistema fue conceptualizado con base en necesidades reales detectadas en el entorno hospitalario, derivadas del levantamiento de requerimientos con administrativo y técnico. Las funcionalidades fueron planteadas de forma modular y escalable, permitiendo que cada componente pudiera ser desarrollado, probado e integrado de forma independiente.

Las funcionalidades clave del sistema abarcaron:

- Agendamiento inteligente de citas médicas (por especialidad, médico, horario disponible)
- Visualización de agenda médica en tiempo real
- Control y gestión de cancelaciones
- Notificaciones automáticas por correo electrónico
- Bloqueo de horarios en tiempo real y liberación de espacios no confirmados
- Visualización de expedientes clínicos por parte del médico
- Historial y estado de citas por parte del tutor

Todas estas funcionalidades se enfocaron en automatizar procesos

manuales que actualmente se gestionan en papel o con herramientas fragmentadas, mejorando significativamente la eficiencia del personal del hospital y la experiencia del paciente.

Durante el análisis inicial, cada funcionalidad fue evaluada en términos de:

- **Implementabilidad técnica** (¿era posible con los recursos y conocimientos disponibles?)
- **Dependencias tecnológicas** (¿se requería software adicional o librerías externas?)
- **Valor para el usuario** (¿aportaba beneficios reales para pacientes o personal?)

Como resultado, se clasificaron los módulos en tres niveles de prioridad: esenciales, importantes y opcionales, permitiendo centrar los esfuerzos en los componentes críticos. Las funcionalidades esenciales (como agendar, visualizar y cancelar citas) fueron completamente desarrolladas, mientras que funcionalidades más avanzadas (como reportes administrativos o configuración dinámica de políticas) fueron parcialmente abordadas o planificadas como mejoras futuras.

Esta estrategia permitió asegurar una cobertura completa de los requerimientos fundamentales sin comprometer la calidad del producto ni exceder los tiempos establecidos.

b) Tecnologías utilizadas.

El stack tecnológico fue seleccionado estratégicamente para maximizar el dominio técnico del equipo, reducir la curva de aprendizaje y garantizar que el sistema pudiera ser desplegado sin restricciones de licenciamiento o infraestructura:

- **Frontend:**
 - **HTML5:** estructura semántica de las vistas.
 - **CSS3:** diseño adaptable y responsivo mediante Grid y Flexbox.
 - **JavaScript:** comportamiento dinámico, validación de formularios y sincronización de horarios.
- **Backend:**
 - **PHP:** lógica de negocio, conexión con la base de datos y validación de datos del lado del servidor.
 - **MySQL:** almacenamiento estructurado de información clínica, de usuarios y citas médicas.
- **Librerías y herramientas externas:**
 - **FullCalendar:** visualización de la agenda médica en formato calendario interactivo.
 - **PHPMailer:** envío automatizado de correos electrónicos para

- confirmaciones, recordatorios y cancelaciones.
- o **Cron Jobs**: automatización de tareas de mantenimiento o recordatorios a través del servidor.
- **Entornos de desarrollo**:
 - o **Visual Studio Code**: principal entorno de codificación por su ligereza y extensibilidad.
 - o **Notepad++**: para ediciones rápidas y visualización ligera de scripts.
 - o **XAMPP**: entorno local para pruebas de PHP y MySQL en fase de desarrollo.

Estas tecnologías fueron totalmente accesibles para los integrantes del equipo, y su adopción no implicó costos adicionales, lo que aumentó la viabilidad del proyecto. A su vez, el equipo ya contaba con conocimientos previos en este stack gracias a asignaturas anteriores (como Desarrollo Web, Bases de Datos y Programación Web), lo cual redujo al mínimo las necesidades de capacitación.

Evaluación de la factibilidad técnica

Criterio Técnico	Evaluación
Cobertura de funcionalidades críticas	Completa
Modularidad del sistema	Alta
Accesibilidad y disponibilidad de tecnologías	Alta
Nivel de conocimiento del equipo	Medio a alto
Necesidad de capacitación externa	Mínima
Licenciamiento y requerimientos legales	Libre uso, sin restricciones
Compatibilidad con infraestructura local	Totalmente compatible (XAMPP)

7.2 Factibilidad operativa.

La factibilidad operativa del sistema fue cuidadosamente evaluada desde la etapa de planeación, y durante su ejecución se pudo comprobar que el equipo contaba con los elementos necesarios para desarrollar y entregar un producto funcional, útil y adaptable a las necesidades del cliente.

a) Recurso humano disponible y capacidades

El proyecto fue desarrollado por un equipo multidisciplinario de cuatro estudiantes, quienes mostramos un compromiso constante y una clara disposición para aplicar los conocimientos técnicos y adquirir nuevas habilidades cuando fue necesario. La formación académica previa en asignaturas como Desarrollo Web, Ingeniería de Software, Bases de Datos y Programación permitió al equipo tener una base sólida desde el

inicio del proyecto.

Cada miembro del equipo aportamos desde su área de experiencia, pero también se fomentó la rotación de tareas menores para fortalecer el aprendizaje colaborativo. Se cubrieron áreas como el diseño de interfaces, codificación del backend, análisis de requerimientos, aseguramiento de la calidad, control de versiones, redacción de documentación técnica y validación con el cliente.

La capacidad del equipo para trabajar de forma autoorganizada, resolver conflictos internos, compartir conocimiento y mantener la motivación fue clave para el cumplimiento de los objetivos de cada sprint. Además, se gestionaron adecuadamente los tiempos entre carga académica, reuniones de equipo y entregas, lo que permitió una ejecución fluida del proyecto.

b) Roles en la metodología Scrum

El equipo adoptó el marco de trabajo Scrum para estructurar y gestionar el desarrollo del sistema. Cada miembro desempeñó un rol específico:

- **Scrum Master – Raziel Trujeque:** Facilitó el cumplimiento de la metodología, eliminó impedimentos para el equipo, promovió la comunicación fluida y organizó las retrospectivas para mejorar continuamente.
- **Product Owner – Armando Tepale:** Representó al cliente, definió el Product Backlog, priorizó funcionalidades y validó los entregables conforme a los requerimientos del Hospital Infantil de las Californias.
- **Equipo de Desarrollo:**
 - **Carlos Vela:** responsable del análisis de requerimientos, implementación de la lógica de backend, pruebas de calidad (QA) y soporte técnico.
 - **Sofía Chavez:** Diseñadora y desarrolladora de frontend, encargada de construir interfaces responsivas, asegurar la experiencia de usuario e integrar las vistas con la lógica de negocio.

Gracias a esta distribución, todos los roles esenciales de Scrum fueron cubiertos sin vacíos, lo que permitió cumplir con los objetivos del proyecto en cada iteración.

c) Plan de comunicación con el cliente y usuarios

Uno de los pilares del éxito operativo fue el establecimiento de un canal de comunicación claro, constante y bidireccional con los representantes del Hospital Infantil de las Californias. Se adoptaron las siguientes prácticas:

- **Reuniones virtuales programadas** mediante plataformas de videollamada (Zoom o Google Meet), especialmente antes de cada Sprint Planning y después de cada Sprint Review.
- **Documentación colaborativa** mediante Google Drive y Notion, donde se compartieron actas, acuerdos, avances y validaciones de requisitos.
- **Seguimiento visual de tareas** con tableros Kanban en Jira, permitiendo al cliente tener visibilidad sobre el avance de los desarrollos.
- **Canales internos de comunicación** a través de Discord, donde el equipo resolvía dudas técnicas, organizaba reuniones internas y compartía avances de forma asincrónica.

Esta estrategia de comunicación permitió involucrar al cliente durante todo el proceso, recolectar retroalimentación continua y realizar ajustes de forma oportuna. También fortaleció la empatía del equipo con las necesidades reales del hospital.

7.3 Análisis FODA

El siguiente análisis FODA permite entender el contexto completo del desarrollo, detectando los factores internos (Fortalezas y Debilidades) y externos (Oportunidades y Amenazas) que impactaron en la ejecución del proyecto:

Fortalezas (F)

- Dominio técnico del equipo en herramientas como PHP, MySQL, HTML, CSS y JavaScript.
- Buena organización interna y adopción efectiva de la metodología Scrum.
- Capacidad de trabajo colaborativo y adaptabilidad al cambio.
- Disponibilidad de herramientas gratuitas y de código abierto.
- Diseño modular del sistema, facilitando la integración de nuevas funcionalidades.

Oportunidades (O)

- Proyecto aplicado en un entorno real, con posibilidades de ser adoptado institucionalmente.
- Capacidad de escalar el sistema para incluir otras áreas como farmacia, laboratorio o historial clínico completo.
- Potencial de mejora continua con nuevas funcionalidades como análisis estadísticos o firma digital de citas.
- Posibilidad de ser utilizado como base para futuros proyectos de investigación o vinculación universitaria.

Debilidades (D)

- Tiempo limitado para desarrollar todas las funcionalidades deseadas.
- Dificultades menores para realizar pruebas reales con usuarios finales debido a restricciones de acceso al entorno hospitalario.

- Algunas validaciones de seguridad y cumplimiento normativo solo fueron implementadas de manera parcial.

Amenazas (A)

- Cambios imprevistos en los requerimientos del cliente durante etapas avanzadas del desarrollo.
- Riesgo de que el sistema no sea adoptado si no se realiza una fase formal de prueba con usuarios reales.
- Descontinuación del mantenimiento si no se asigna personal interno del hospital o se formaliza su implementación.

Estrategias derivadas del FODA

- **FO (Fortalezas + Oportunidades):** Aprovechar el dominio técnico del equipo y la estructura del sistema para proponer su adopción en otras áreas del hospital. Presentar una demostración funcional y elaborar un plan de integración por fases.
- **DO (Debilidades + Oportunidades):** Establecer como fase siguiente la capacitación del equipo en herramientas como Power BI, Chart.js o librerías de gráficos para mejorar los reportes estadísticos.
- **FA (Fortalezas + Amenazas):** Aplicar el enfoque ágil y modular del sistema para adaptarse con rapidez a cambios de requerimientos. Documentar exhaustivamente para facilitar futuras actualizaciones.
- **DA (Debilidades + Amenazas):** Implementar una fase de pruebas piloto con personal real del hospital para validar la utilidad del sistema y recolectar retroalimentación antes de su posible adopción definitiva.

8.0 Análisis de Riesgos

8.1. Identificación de los riesgos

Durante el desarrollo de este proyecto se pudieron encontrar y estimar posibles riesgos, la mayoría de estos sucedieron, pero se pudieron de sobrellevar de la mejor manera que pudimos.

Entre los riesgos principales destacaron:

- Retrasos en las entregas, si bien al inicio del proyecto estuvimos atrasados, ya que se firmaron los requerimientos nosotros cumplimos con las entregas estipuladas por el Hospital.
- Fallos en el servidor local, las mayores complejidades que encontramos en este punto era que para todos era la primera vez trabajando en un servidor así, y la instalación del php fue la que más complicaciones nos dio, tratando de averiguar que fallaba.
- Errores en el código backend, aquí si hubo fallos más que nada en la lógica de la programación, pero nada que no se esperara y en trabajando en equipo pudimos detectar dichos fallos.
- Falta de comunicación en el equipo, si bien se esperaba que

la comunicación si fuera un pequeño problema, conforme fue avanzando el proyecto este se volvió más grande, a tal punto que escaló a que el coordinador de proyecto tuviera que intervenir

- Desacuerdos en decisiones técnicas, en general este no fue un problema ya que todos los integrantes estuvieron en las juntas con el Hospital y teníamos una idea muy similar en cómo se iba a llevar a cabo el proyecto y en las juntas hechas solo con el equipo llegábamos a acuerdos sin más problemas, todo en bien del proyecto.
- Fallos al subir el HOST, los fallos en el local fueran más que nada en la instalación de las herramientas que utilizamos, pero al momento de subirlo al host otorgado por el hospital presentamos fallos en el calendario ya que no se visualizaba como debería en una de nuestros menús.
- Retrasos por entregas de equipos complementarios, un riesgo que no predijimos al principio como tal fue que nuestro proyecto dependía de otra parte, y que el equipo del cual dependemos sus entregas si fueron atrasadas lo que provocó que tuviéramos que hacer trabajo que en teoría no nos correspondía para que nuestras entregas no se vieran afectadas.
- Baja disponibilidad de integrantes, un riesgo que sabíamos que podía ser muy crítico era la disponibilidad de todos, ya que fuera de las materias que llevábamos, todos los integrantes trabajan y adaptarnos la mayoría a los horarios resultaba algo complicado, por suerte gran parte de las veces acordábamos horarios muy tarde para las reuniones entre nosotros.
- Mal diseño de base de datos, fuera de que fuera un mal diseño se creó una base de datos en la que con el paso del tiempo nos dimos cuenta de que le faltaba una tabla que agregar, no fue gran inconveniente porque nos dimos cuenta a tiempo.
- Errores en pruebas finales, por sprint detectamos errores y en las presentaciones al Hospital también, sin embargo teníamos la capacidad para decir que era lo que estaba pasando y resolverlo en su debido momento.

8.2. Caracterización de los riesgos

- Retraso en las entregas: Crítico. Provocado por demoras en tareas clave del sprint.
- Fallo en servidor local: Alto. Causó pérdida de tiempo y posibles daños a archivos.
- Errores backend: Medio. Afectó la implementación funcional.
- Falta de comunicación: Alto. Comprometió el avance del proyecto.
- Desacuerdos técnicos: Bajo. Se logró llegar a un acuerdo fácilmente.

- Fallos al subir al HOST: Crítico. Pérdida de archivos.
- Retraso de equipo externo: Crítico. Aumentó la carga al equipo.
- Baja disponibilidad: Medio. Redujo el avance general.
- Mal diseño de BD: Bajo. Se detectó a tiempo
- Errores finales: Crítico. Amenazó el cumplimiento del sprint.

8.3. Acciones para enfrentar los riesgos

Prevención:

Desde el inicio del proyecto sabíamos que debíamos tomar acciones para la prevención de ciertos riesgos a los cuales nos podríamos enfrentar

- Planificar tareas y revisar avances anticipadamente.
- Respalidar archivos en la nube o usar entornos alternativos.
- Establecer reuniones semanales con canales claros.
- Acordar decisiones técnicas en grupo.
- Mejor comunicación entre equipos relacionados.
- Anticipar bajas y repartir tareas con flexibilidad.
- Revisar diseño de BD con el equipo.
- Validar funcionalidades antes de entregar.

Recuperación:

En lo general podemos decir que contar con un plan de prevención fue de mucha ayuda ya que el tener respaldos de las cosas que íbamos haciendo hizo que la recuperación fuera más sencilla

- Extender plazos o reasignar tareas si es necesario.
- Restaurar respaldos y cambiar entorno si falla.
- Reunión urgente para reorganizar tareas.
- Escalar el conflicto al Scrum Master si no hay consenso.
- Subir archivos con respaldo verificado.
- Planificar el sprint afectado.
- Redistribuir carga tras confirmar disponibilidad.
- Rediseñar estructura de BD si es necesario.

9. Desarrollo por Sprints.

9.1. Sprint 1

a) Sprint Planning

Objetivo del Sprint: Construir el flujo completo de agendamiento de citas, incluyendo el registro de datos del paciente, selección de médico por especialidad, fechas y horarios disponibles, con validaciones en tiempo real.

Duración del Sprint: 2 semanas.

Horas disponibles por integrante: 20 horas × 4 integrantes = 80 horas.

User Stories agrupadas en este Sprint: US01, US02, US04, US05.

Tarea	Detalle específico	Estimación (horas)
Diseño de formulario de agendamiento	Estructura HTML, validaciones con JavaScript, integración visual	6 h
Validación del número de expediente	Verificación contra la base de datos de pacientes	4 h
Filtro y consulta de médicos por especialidad	Listado dinámico por especialidad, integración con BD	6 h
Calendario de fechas (FullCalendar)	Restricción de días inválidos, habilitación por disponibilidad	6 h
Visualización y selección de horarios	Mostrar horarios libres, bloquear solapamientos	6 h
Pruebas unitarias del módulo de agendamiento	Casos límite, validaciones en frontend y backend	4 h
Documentación y comentarios en código	Comentarios, estructura clara y manual de funcionamiento	2 h

Horas estimadas: 34 horas

b) Descripción de User Stories (Sprint 1)

User Stories cubiertas:

US01 a US05 - Agendamiento integral.

- **Criterios de aceptación combinados:**

- El tutor debe poder ingresar un expediente válido y visualizar automáticamente los datos del paciente.

- El sistema debe filtrar médicos según especialidad elegida.
- Solo se deben permitir fechas futuras y disponibles.
- Los horarios deben bloquearse si ya fueron reservados.

- **Flujos resumidos:**

1. Tutor ingresa expediente.
2. Se muestran datos del paciente y especialidades.
3. El tutor selecciona especialidad → se cargan médicos disponibles.
4. El tutor elige la fecha del calendario.
5. El sistema valida disponibilidad.
6. El tutor selecciona hora → sistema bloquea ese horario.
7. Se confirma la cita y se almacena en BD.

- **Casos de prueba importantes:**

- Expediente inexistente → error.
- Especialidad sin médicos → mensaje informativo.
- Fecha inválida → bloqueada.
- Horario ya ocupado → no aparece disponible.

c) Desarrollo de User Stories

- (Se completará con evidencia técnica del equipo, fragmentos de código, estructura de base de datos y capturas de pantalla.)

```
USE hospital_db;
CREATE TABLE especialidades (
  id_especialidad INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(100) NOT NULL,
  duracion_minutos INT NOT NULL,
  estado ENUM('activo', 'inactivo') DEFAULT 'activo'
);
CREATE TABLE servicios (
  id_servicio INT PRIMARY KEY AUTO_INCREMENT,
  id_especialidad INT,
  nombre VARCHAR(100) NOT NULL,
  precio DECIMAL(10,2) NOT NULL,
  descripcion TEXT,
  estado ENUM('activo', 'inactivo') DEFAULT 'activo',
  FOREIGN KEY (id_especialidad) REFERENCES especialidades(id_especialidad)
  ON DELETE SET NULL
  ON UPDATE CASCADE
);
CREATE TABLE tutor (
  id_tutor INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  pais_residencia VARCHAR(100),
  estado_residencia VARCHAR(100),
  email VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) CHARACTER SET utf8mb4 COLLATE utf8mb4_bin NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE expediente (
  id_expediente INT AUTO_INCREMENT PRIMARY KEY,
  id_tutor INT,
  nombre VARCHAR(50) NOT NULL,
  apellido_paterno VARCHAR(50) NOT NULL,
  apellido_materno VARCHAR(50) NOT NULL,
  servicio_medico VARCHAR(50) NOT NULL,
  derechohabiente VARCHAR(50) NOT NULL,
  sexo ENUM('masculino', 'femenino') NOT NULL,
  fecha_nacimiento DATE NOT NULL,
  curp VARCHAR(20) NOT NULL,
  pais_nacimiento VARCHAR(100) NOT NULL,
  estado_nacimiento VARCHAR(100) NOT NULL,
  ciudad_nacimiento VARCHAR(100) NOT NULL,
  pais_reside VARCHAR(100) NOT NULL,
  estado_reside VARCHAR(100) NOT NULL,
  ciudad_reside VARCHAR(100) NOT NULL,
  asistencia_escolar ENUM('si', 'no') NOT NULL,
  nivel_estudio VARCHAR(100) NOT NULL,
  parentesco VARCHAR(20) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_tutor) REFERENCES tutor(id_tutor)
  ON DELETE SET NULL
  ON UPDATE CASCADE
);
CREATE TABLE medicos (
  id_medico INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(50) NOT NULL,
  apellido_paterno VARCHAR(50) NOT NULL,
  apellido_materno VARCHAR(50),
  escuela_estudios VARCHAR(100),
  cedula_profesional VARCHAR(50) UNIQUE,
  id_especialidad INT,
  foto_url VARCHAR(255),
  FOREIGN KEY (id_especialidad) REFERENCES especialidades(id_especialidad)
  ON DELETE SET NULL
  ON UPDATE CASCADE
);
CREATE TABLE horarios_medico (
  id_horario INT PRIMARY KEY AUTO_INCREMENT,
  id_medico INT NOT NULL,
  dia_semana ENUM('lunes', 'martes', 'miércoles', 'jueves', 'viernes', 'sábado') NOT NULL,
  hora_inicio TIME NOT NULL,
  hora_fin TIME NOT NULL,
  FOREIGN KEY (id_medico) REFERENCES medicos(id_medico)
  ON DELETE CASCADE
  ON UPDATE CASCADE
);
CREATE TABLE citas (
  id_cita INT PRIMARY KEY AUTO_INCREMENT,
  id_tutor INT,
  id_paciente INT,
  id_medico INT,
  id_especialidad INT,
  id_servicio INT,
  fecha DATE NOT NULL,
  hora TIME NOT NULL,
  estado ENUM('pendiente', 'aprobada', 'rechazada', 'cancelada') DEFAULT 'pendiente',
  fecha_creacion DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_tutor) REFERENCES tutor(id_tutor)
  ON DELETE SET NULL
  ON UPDATE CASCADE,
  FOREIGN KEY (id_paciente) REFERENCES expediente(id_expediente)
  ON DELETE CASCADE
  ON UPDATE CASCADE,
  FOREIGN KEY (id_medico) REFERENCES medicos(id_medico)
  ON DELETE SET NULL
  ON UPDATE CASCADE,
  FOREIGN KEY (id_especialidad) REFERENCES especialidades(id_especialidad)
  ON DELETE SET NULL
  ON UPDATE CASCADE,
  FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
  ON DELETE SET NULL
  ON UPDATE CASCADE
);
CREATE TABLE cancelaciones (
  id_cancelacion INT PRIMARY KEY AUTO_INCREMENT,
  id_cita INT NOT NULL,
  fecha_cancelacion DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_cita) REFERENCES citas(id_cita)
  ON DELETE CASCADE
  ON UPDATE CASCADE
);
```

d) Pruebas de funcionalidades

- **US01:** Verificación de datos del paciente a través del expediente.

**HOSPITAL
INFANTIL**

DE LAS CALIFORNIAS

Acceso Usuario

Número de expediente

Ingresar

[¿Olvidaste tu contraseña?](#)

- **US02:** Mostrar médicos disponibles para agendar según especialidad.



**HOSPITAL
INFANTIL**



DE LAS CALIFORNIAS

Registrar Nuevo Doctor

Nombre

Apellido Paterno

Apellido Materno

Cédula Profesional

Especialidad

Seleccione una especialidad

Servicio

Seleccione un servicio

Escuela de Estudios

Guardar Doctor

Registrar Horarios Semanales

Dashboard

Doctores

Especialidades

Servicios

Agenda de Citas

Citas Disponibles

Pacientes

Reportes

Nuevo Administrador

Cerrar sesión

Lista de doctores registrados

Nombre	Apellido	Cédula	Especialidad	Servicio	Acciones
Andrea	Tepale	1281234	Pediatría	Consulta	Editar
Jesus Alfredo	Fuentes	5218549	Terapeuta	Sin servicio	Editar
Juan	Ramirez	5197843	Pediatría	Sin servicio	Editar
Francisco	Martinez	9874561	Cardiología	Sin servicio	Editar
andrea	chavez	1279990	Cardiología	Sin servicio	Editar

- **US04 y US05:** Permitir la selección de fecha y hora según disponibilidad.

jue 29/5		vie 30/5		sab 31/5	
8:00 - 8:30 - Consulta					
8:30 - 9:00 - Consulta					
9:00 - 9:30 - 9:00 - 9:30 -		9:00 - 9:30 - Consulta		9:00 - 9:30 - Consulta	
9:30 - 10:00 - Consulta					
10:00 - 10:30 - Consulta		10:00 - 10:30 - Consulta		10:00 - 10:30 - Consulta	
10:30 - 11:00 - Consulta					
11:00 - 11:30 - 11:00 - 11:30 -		11:00 - 11:30 - Consulta		11:00 - 11:30 - Consulta	
11:30 - 12:00 - Consulta					
12:00 - 12:30	12:00 - 12:30			12:00 - 12:30	12:00 - 12:30
12:30 - 13:00	12:30 - 13:00			12:30 - 13:00	12:30 - 13:00
13:00 - 13:30	13:00 - 13:30			13:00 - 13:30 - Consulta	13:00 - 13:30 - Consulta
13:30 - 14:00	13:30 - 14:00			13:30 - 14:00 - Consulta	13:30 - 14:00 - Consulta
14:00 - 14:30	14:00 - 14:30			14:00 - 14:30 - Consulta	14:00 - 14:30 - Consulta
14:30 - 15:00	14:30 - 15:00			14:30 - 15:00 - Consulta	14:30 - 15:00 - Consulta
15:00 - 15:30	15:00 - 15:30			15:00 - 15:30 - Consulta	15:00 - 15:30 - Consulta
15:30 - 16:00	15:30 - 16:00			15:30 - 16:00 - Consulta	15:30 - 16:00 - Consulta
16:00 - 16:30	16:00 - 16:30			16:00 - 16:30 - Consulta	16:00 - 16:30 - Consulta
16:30 - 17:00	16:30 - 17:00			16:30 - 17:00 - Consulta	16:30 - 17:00 - Consulta

- **US08:** Impedir agendar si no hay horarios disponibles.

- **US11:** Visualizar el estado de cada cita (disponible, ocupada, cancelada).

Detalle de Cita	Detalle de Cita
Paciente: Luisito	Paciente: Sin asignar
Médico: Dr. Andrea	Médico: Dr. Andrea
Fecha: 2025-05-29	Fecha: 2025-05-29
Hora: 08:00:00	Hora: 10:00:00
Servicio Medico: -	Servicio Medico: -
Derechoabiente: ocupado	Derechoabiente: disponible
Estado: ocupado	Estado: disponible

- **US12:** Permitir cancelación por parte del usuario tutor.

 Parentesco:	Hijo
 Teléfono:	664123123
 Email:	schavez67@uabc.edu.mx

9.2. Sprint 2

a) Sprint Planning

Objetivo del Sprint: Implementar el módulo de visualización, historial y cancelación de citas. El tutor debe poder consultar el estado actual de sus citas, cancelarlas si es necesario y acceder a un historial completo.

Duración del Sprint: 2 semanas.

User Stories agrupadas en este Sprint: US06, US10, US12, US14.

Estimación total del Sprint: 16 Story Points.

Tarea	Detalle específico	Estimación (horas)
Visualización de citas agendadas	Listado dinámico filtrado por paciente	5 h
Consulta de detalles de cita	Mostrar información completa de cita	5 h
Cancelación de cita	Confirmación, bloqueo y envío de notificación por correo	6 h
Implementación de historial de citas	Listado con estado: realizadas, canceladas, pendientes	6 h
Pruebas funcionales y validación con cliente	Escenarios de cancelación, carga condicional, errores esperados	4 h

Horas estimadas: 26 horas.

b) Descripción de User Stories (Sprint 2)

User Stories cubiertas:

US06, US10, US12, US14 - Gestión post-agendamiento.

- **Criterios de aceptación combinados:**

- El tutor puede consultar sus citas actuales y pasadas.
- Puede cancelar cualquier cita pendiente con confirmación.
- El sistema debe registrar cada cambio y actualizar el estado de la cita.

- **Flujos principales:**

1. Tutor accede a módulo de citas agendadas.
2. Selecciona cita y ve sus detalles.
3. Si desea cancelarla, confirma la acción.
4. Se actualiza estado a "cancelada" y se registra en historial.
5. Historial muestra todas las citas del paciente en orden cronológico.

- **Casos de prueba importantes:**

- Cita pasada → no editable.
- Cita pendiente → opción de cancelación activa.
- Cita cancelada → aparece en historial con estado correcto.

c) Desarrollo de User Stories

[illegible]

```
<!DOCTYPE html>
<html lang=es>
<head>
    <meta charset=utf-8>
    <meta content=width=device-width, initial-scale=1 name=viewport>
    <link href=/images/logo/hospital_msc_vertical.png rel=shortcut icon>
    <title>Citas Disponibles | Hospital Infante de Lituia</title>
    <link href=https://cdn.jsdelivr.net/npm/bootstrap@5.1/dist/css/bootstrap.min.css rel=stylesheet>
    <link href=https://cdn.jsdelivr.net/npm/fullcalendar@6.1.9/index.global.min.css rel=stylesheet>
    <link href=https://fonts.googleapis.com/css?family=Montserrat&display=block rel=stylesheet>
    <script src=https://cdn.jsdelivr.net/npm/fullcalendar@6.1/index.global.min.js></script>
    <script src=https://cdn.jsdelivr.net/npm/bootstrap@5.3/dist/js/bootstrap.bundle.min.js></script>
    <style>
        body {
            background-color:#f0f0f0;
            font-family: 'Montserrat', sans-serif;
            animation: fadeIn 0.5s ease-in;
        }
        header-gradient {
            background: linear-gradient(18deg, #d7d7d7, #e7e7e7);
            color: white;
            border-radius: 12px;
        }
        .card {
            border-radius: 12px;
            box-shadow: 0 4px 12px rgba(0, 0, 0, 0.05);
            transition: transform 0.3s ease;
        }
        .card:hover {
            transform: scale(1.01);
        }
        .btn-outline-light:hover {
            background-color: white;
            color: #d7d7d7;
        }
        #calendar {
            border-radius: 8px;
            overflow: hidden;
            background-color: white;
        }
        .modal-header.bg-success {
            background-color: #38b28a !important;
        }
        .fc-event-disposable {
            background-color: #d7d7d7 !important;
            border-color: #d7d7d7 !important;
            color: white !important;
            font-weight: bold;
        }
        .fc-event-scoped {
            background-color: #d7d7d7 !important;
            border-color: #d7d7d7 !important;
            color: white !important;
            font-weight: bold;
        }
        @keyframes fadeIn {
            from { opacity: 0; transform: translateX(100%); }
            to { opacity: 1; transform: translateX(0); }
        }
        @media (max-width: 768px) {
            .fc-toolbar-title {
                font-size: 1rem !important;
            }
            .fc-button {
                font-size: 0.8rem !important;
                padding: 0.2rem 0.5rem !important;
            }
            .fc-timeline-day-totals {
                font-size: 0.7rem !important;
            }
            .fc-col-cell-header.col-cell, .fc-day-today-number {
                font-size: 0.75rem !important;
            }
            header-gradient {
                flex-direction: column !important;
                text-align: center;
            }
        }
    </style>
</head>
<body>
    <div class=Container-fluid py-4 px-3 px-sm=0>
        <div class=header-gradient h-100 flex-column flex-md-row justify-content-between align-items-center>
            <div class=fas fa-hospital class=me-3 src=/images/logo-1.png style=height: 40px;></div>
            <div class=h1-h2 text-mw=100 text-sm=100 flex-calendar-center me-2>Lituia</div>
            <div class=mb-0>Consulta las citas disponibles y agenda con tu correo.</div>
            <div>
                <div class=fas fa-envelope-alt class=icon-text=md md=sm md=lg>
                    <input type=email hospital, class=me-3 src=/images/logo-1.png style=height: 40px;>
                </div>
                <div class=w-100 text-mw=100 text-sm=100 text-md=100 text-lg=100>
                    <button href=/agendar/>Agendar ahora</button>
                </div>
            </div>
        </div>
    </div>
```

d) Pruebas de funcionalidades

- **US03:** Permitir al usuario seleccionar un médico específico del listado.

Médico

Todos

Todos

Andrea Tepale

Jesus Alfredo Fuentes

Juan Ramirez

Francisco Martinez

andrea chavez

Raziel Trujeque

Pedro Ramirez

Alejandro Perez

Ricardo Perez

- **US07:** Restringir agendamientos para citas que ocurren el mismo día.



Cita agendada ya no aparece

- **US09:** Mostrar todas las citas agendadas por el usuario



**HOSPITAL
INFANTIL**
 DE LAS CALIFORNIAS


Citas Asignadas
 Consulta las citas asignadas y agenda con un tutor.

[Agendar Cita](#)
[Cerrar sesión](#)

Información de tu Cita

 Paciente:	Anita
 Derechohabiente:	SI
 Servicio Médico:	ISSSTE
 Genero:	femenino
 Fecha de Nacimiento:	2014-02-17
 CURP:	GOSAI40217MDFMNA03
 Fecha Cita:	2025-05-29
 Hora Inicio:	08:30:00
 Hora Fin:	09:00:00
 Médico:	Andrea

Información del Tutor

 Nombre:	Sofia Chavez Munguia
---	----------------------

- **US10:** Actualizar el estado de la cita después de agendar o cancelar.

PEDIATRIA		CARDIOLOGIA		CIRUGIA	
8:30 - 9:00 - Pediatría con					
9:00 - 9:30 - Pediatría con		9:00 - 9:30 - Cardiología con			
		10:00 - 10:30 - Cardiología con		10:00 - 10:30 - Pediatría con	
10:30 - 11:00 - Pediatría con					
11:00 - 11:30 - Cirujano con	11:00 - 11:30 - P	11:00 - 11:30 - Cirujano con		11:00 - 11:30 - Pediatría con	
11:30 - 12:00 - Pediatría con					
12:00 - 12:30 - P	12:00 - 12:30 - I			12:00 - 12:30 - C	12:00 - 12:30 - I
	12:30 - 13:00 - I				12:30 - 13:00 - I
12:30 - 13:00 - P	12:30 - 13:00 - I				
13:00 - 13:30 - P	13:00 - 13:30 - I			13:00 - 13:30 - Pediatría con	
	13:30 - 14:00 - I			13:30 - 14:00 - Pediatría con	
13:30 - 14:00 - P	13:30 - 14:00 - I			14:00 - 14:30 - Pediatría con	
14:00 - 14:30 - P	14:00 - 14:30 - I			14:30 - 15:00 - Pediatría con	
14:30 - 15:00 - P	14:30 - 15:00 - I			15:00 - 15:30 - Pediatría con	
15:00 - 15:30 - P	15:00 - 15:30 - I			15:30 - 16:00 - Pediatría con	
15:30 - 16:00 - P	15:30 - 16:00 - I			16:00 - 16:30 - Pediatría con	
16:00 - 16:30 - P	16:00 - 16:30 - I			16:30 - 17:00 - Pediatría con	
16:30 - 17:00 - P	16:30 - 17:00 - I				

La cita después de cancelar vuelve a aparecer en verde

9.3. Sprint 3

a) Sprint Planning

Objetivo del Sprint: Consolidar el módulo administrativo con reportes, edición de citas y sincronización en tiempo real para evitar solapamientos entre usuarios.

Duración del Sprint: 2 semanas.

User Stories agrupadas en este Sprint: US15, US16, US17, US20.

Estimación total del Sprint: 17 Story Points.

Tarea	Detalle específico	Estimación (horas)
Generación de reportes administrativos	Filtros por especialidad, médico, fecha y visualización en tablas y gráficos	7 h
Modificación de citas existentes	Validación de edición, notificación de cambio	6 h
Implementación de bloqueo de horario inmediato	Lógica de bloqueo temporal, BD, y frontend dinámico	6 h
Sincronización de horarios en tiempo real	Integración de WebSocket o polling	6 h
Pruebas de simultaneidad y colisiones	Validación con múltiples usuarios concurrentes	4 h

Horas estimadas: 29 horas

b) Descripción de User Stories (Sprint 3)

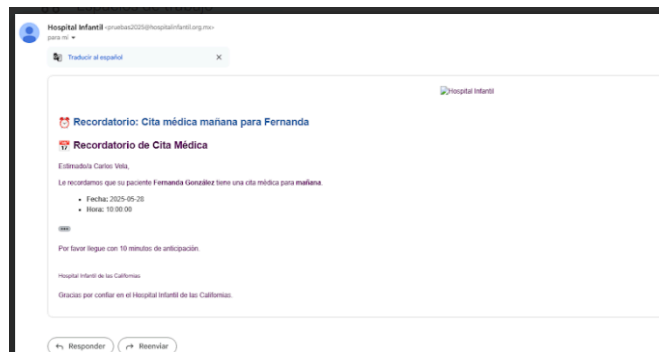
User Stories cubiertas:

US15, US16, US17, US20 - Funcionalidades administrativas avanzadas

- **Criterios de aceptación combinados:**
 - El administrador debe poder generar reportes visuales según criterios de búsqueda.
 - Se debe permitir la edición de citas si están activas.

d) Pruebas de funcionalidades

- **US13:** Enviar recordatorios por correo electrónico al tutor y/o paciente.



- **US14:** Mostrar el historial completo de citas por usuario.

Fecha Inicio

dd/mm/aaaa

Fecha Fin

dd/mm/aaaa

Médico

Todos

Estado

Todos

Resultados **Exportar Excel** **Exportar PDF**

Fecha	Hora	Médico	Especialidad	Paciente	Tutor	Estado
-------	------	--------	--------------	----------	-------	--------

- **US19:** Actualizar disponibilidad del calendario automáticamente al agendar o cancelar.



10. Estimación del costo del proyecto

La estimación del costo del proyecto se realizó con base en la metodología ágil Scrum, utilizando como unidad de medida los Story Points asignados a cada historia de usuario (US), y convirtiéndolos en horas de trabajo estimadas por Sprint y por rol (Scrum Master, Product Owner y Equipo de Desarrollo).

Se consideraron las siguientes variables para esta estimación:

- Duración de cada Sprint: 2 semanas
- Horas laborales por persona por semana: 10 horas
- Costo estimado por hora de cada rol (valor teórico):
 1. Scrum Master: \$150 MXN/hora
 2. Product Owner: \$130 MXN/hora
 3. Desarrolladores: \$100 MXN/hora
- Total de integrantes: 4 (1 Scrum Master, 1 Product Owner, 2 Developers)
- Planeación y documentación: estimada en 15% del tiempo total del proyecto.

10.1. Estimación basada en Story Points y Sprints.

Durante el desarrollo, el backlog fue priorizado y dividido en Sprints con base en los Story Points (SP) asignados. Estos puntos reflejan el esfuerzo y complejidad de cada historia de usuario.

Sprint	Total de SP estimados	Horas estimadas (equivalente aprox.)
Sprint 1	20 SP	80 horas
Sprint 2	25 SP	100 horas
Sprint 3	20 SP	120 horas
<i><u>Total estimado</u></i>	75 SP	300 horas

La conversión estimada fue de 1 SP $\approx$ 4 horas de trabajo técnico (programación, pruebas, integración).

10.2. Distribución de horas por rol

La siguiente tabla muestra la estimación de horas dedicadas por cada rol en el proyecto, incluyendo las actividades de desarrollo, planeación, coordinación y pruebas.*Costo promedio ponderado considerando participación de todos los roles.*Costo promedio ponderado considerando participación de todos los roles.

Rol	Actividades principales	Horas estimadas	Costo por hora	Subtotal estimado
Scrum Master	Facilitación de reuniones, resolución de bloqueos, seguimiento de equipo	40 horas	\$150 MXN	\$6,000 MXN
Product Owner	Priorización de backlog, validación de entregables, comunicación con el cliente	35 horas	\$130 MXN	\$4,550 MXN
Developer 1 (Backend + QA)	Desarrollo de funcionalidades, conexión con base de datos, pruebas	100 horas	\$100 MXN	\$10,000 MXN
Developer 2 (Frontend + UX)	Diseño de interfaz, integración visual, responsividad	100 horas	\$100 MXN	\$10,000 MXN
Planeación y documentación	Reuniones de planificación, redacción de reportes y	25 horas	\$120 MXN*	\$3,000 MXN

	documentación del sistema			
--	---------------------------	--	--	--

10.3. Estimación total del proyecto.

Categoría

Subtotal

Trabajo técnico (desarrollo)

\$24,550 MXN

Planeación y documentación

\$3,000

MXN

Costo total estimado

\$27,550 MXN

Este valor representa una estimación basada en el desarrollo parcial del sistema (75 SP). Dado que el alcance completo proyectado (con funcionalidades adicionales como reportes gráficos, historial clínico extendido y gestión avanzada de usuarios) puede llegar a 120 SP, se puede proyectar un costo total estimado para el sistema completo de aproximadamente:

Proyección a sistema completo: 120 SP $\approx$ \$44,080 MXN

10.4. Conclusión de la estimación.

El sistema desarrollado representa una inversión estimada de entre \$27,000 y \$45,000 MXN, dependiendo del grado de cobertura funcional que se pretenda alcanzar en versiones posteriores. Esta estimación permite al cliente (Hospital Infantil de las Californias) tener una referencia económica realista si desea contratar un servicio profesional de desarrollo, mantenimiento o ampliación del sistema.

Además, sirve como guía para futuras propuestas de financiamiento, convenios con otras áreas del hospital o integración con otros sistemas.

11. Conclusiones.

El desarrollo del sistema de Gestión de Citas Médicas para el Hospital Infantil de las Californias fue mucho más que un proyecto académico; fue una experiencia integral que nos permitió aplicar de manera directa los conocimientos adquiridos durante nuestra formación, enfrentar situaciones reales y crecer tanto técnica como humanamente. Este proyecto nos enseñó el verdadero significado de ser estudiantes de ingeniería: aprender haciendo, fallar y corregir, colaborar, adaptarnos y aportar valor real.

Aplicación práctica de conocimientos teóricos.

Uno de los aprendizajes más valiosos fue comprender cómo los conocimientos vistos en distintas materias convergen en un solo proyecto. Aplicamos principios de Ingeniería de Software, desarrollo web, bases de datos, modelado UML, metodologías ágiles, diseño de interfaces y pruebas. Cada concepto cobró sentido cuando lo pusimos en práctica para resolver una necesidad específica del cliente: ordenar y automatizar el sistema de citas médicas.

A través del uso de Scrum, pudimos organizarnos en Sprints, definir prioridades, estimar esfuerzos con Story Points, mantener una visión del backlog, realizar retrospectivas y presentar resultados reales en cada entrega. Así, las clases dejaron de ser sólo teoría y se convirtieron en herramientas para construir algo útil, tangible y funcional.

Cómo fue trabajar en el proyecto.

Trabajar en este proyecto fue, sin exagerar, como vivir una pequeña versión del mundo profesional. Desde el primer día asumimos que no bastaba con entregar código; debíamos diseñar una solución pensando en el usuario, el contexto, la viabilidad técnica y el cumplimiento de objetivos concretos.

La relación con el Hospital Infantil nos exigió ser responsables, organizados y empáticos. Ellos tenían un problema real, y nosotros asumimos el reto de resolverlo con lo que sabíamos, aprendiendo lo que no sabíamos sobre la marcha. No era suficiente que algo “funcionara”: tenía que ser claro, útil y fácil de usar para el personal del hospital y los tutores de los pacientes.

Tuvimos que adaptar nuestro lenguaje para comunicar ideas técnicas a personas no técnicas, realizar entrevistas para levantar requerimientos, responder ante cambios de último momento, rediseñar algunas funciones cuando no eran entendidas del todo, y mantener una relación profesional a pesar de ser estudiantes. Esa interacción fue clave para entender que, en la ingeniería, la tecnología siempre debe estar al servicio de las personas.

Dificultades encontradas y cómo las superamos.

Durante el desarrollo enfrentamos múltiples retos. Uno de los primeros fue la falta de experiencia en ciertos temas, como la generación de reportes visuales o la validación de formularios con múltiples reglas. Afortunadamente, lo enfrentamos como equipo, investigando, probando librerías y ayudándonos mutuamente. Aprendimos a buscar soluciones con documentación oficial, foros, videos y sobre todo con prueba y error.

Otro obstáculo importante fue el tiempo limitado, especialmente cuando se cruzaron entregas de otras materias. En esas semanas la productividad bajó, pero gracias a la planificación por Sprints y la distribución de tareas, logramos salir adelante. Acordamos redistribuir carga, reorganizar prioridades y hasta hacer sesiones colaborativas fuera del horario habitual para recuperar terreno perdido.

También hubo desafíos en la interpretación de los requerimientos del cliente. En más de una ocasión desarrollamos funcionalidades que no eran exactamente lo que el hospital necesitaba. Esto nos hizo darnos cuenta de lo importante que es validar los requerimientos, no asumir cosas y comunicar de forma constante. A partir de esas experiencias, empezamos a usar prototipos rápidos y confirmaciones por escrito antes de codificar.

Satisfacción por la aceptación del cliente.

Uno de los momentos más satisfactorios fue cuando el hospital nos dio su retroalimentación y expresó que el sistema cubría justo las necesidades que tenían. La posibilidad de que un sistema que diseñamos como estudiantes pueda ser usado en un entorno real nos dio una sensación de orgullo y validación que no se obtiene en una tarea común.

Ver cómo una persona externa, ajena al mundo académico, usa tu sistema y dice que le resulta útil, es algo que refuerza la motivación y cambia tu forma de ver la carrera. Ya no es solo una materia que pasas; es una solución que entregas y que podría ayudar a mejorar la atención médica de niños y niñas.

Trabajo en equipo y crecimiento personal.

Si algo nos llevamos también de este proyecto es el valor del trabajo en equipo. Desde el primer Sprint cada integrante asumió un rol definido, pero con el paso del tiempo aprendimos que el éxito no dependía solo de cumplir lo asignado, sino de trabajar en conjunto, ayudar al que se retrasaba, compartir avances, comunicarse con claridad y tener una meta común.

Hubo desacuerdos, momentos de tensión y diferencias de criterio, como en cualquier proyecto real. Pero aprendimos a escuchar, a negociar y a priorizar el avance del equipo por encima del ego personal. Ese tipo de aprendizaje no se enseña en un aula: se vive.

12. Referencias.

1. Aguilar, R. (2021). Metodologías Ágiles: SCRUM aplicado a

proyectos reales. OpenWebinars. <https://openwebinars.net/>

2. Basarte, A. (2022). Bases de datos relacionales con MySQL y PHP. Apuntes.de. <https://apuntes.de/>
3. Beltrán, L. (2023). FullCalendar JS: Integración en sistemas médicos. Blog DevSoft. <https://fullcalendar.io/docs>
4. Beltrán, L. (2023). Buenas prácticas de seguridad en aplicaciones web. Kaspersky Latinoamérica. <https://latam.kaspersky.com/resource-center/>
5. Bravo, J. (2021). Proyectos ágiles con estudiantes: implementación de Scrum en contextos académicos reales. Revista de Innovación Educativa. <https://rie.universidad.edu.mx/>
6. Cordero, J. (2023). Diseño de bases de datos relacionales para sistemas web. Editorial Alfaomega.
7. Damián, C. (2021). Estimación de proyectos de software utilizando Story Points y Planning Poker. Universidad de Piura. <https://www.udep.edu.pe/>
8. González, J. (2020). Diseño de interfaces web accesibles y responsivas. Blog de Desarrollo Web. <https://www.desarrolloweb.com/>
9. Hernández, M. (2019). Pruebas de software: tipos, ejemplos y buenas prácticas. Revista Ingeniería Informática. <https://revistas.unal.edu.co/>
10. INEGI. (2020). Clasificación Nacional de Ocupaciones y Perfiles Tecnológicos. <https://www.inegi.org.mx/>
11. INAI. (2017). Guía para la Protección de Datos Personales en el Desarrollo de Aplicaciones. Instituto Nacional de Transparencia. <https://home.inai.org.mx/>
12. MDN Web Docs. (2024). Documentación de HTML, CSS y JavaScript. Mozilla. <https://developer.mozilla.org/es/>
13. Mora, P. (2020). Análisis FODA aplicado a sistemas informáticos. Revista de Ciencias Empresariales. <https://revistas.tec.mx/>
14. MySQL. (2023). Guía oficial de MySQL 8.0. <https://dev.mysql.com/doc/refman/8.0/es/>
15. Open Webinars. (2022). Curso práctico de desarrollo web

profesional con PHP y JavaScript. <https://openwebinars.net/>

16. Ortega, I. (2020). Uso de Notion y Discord para proyectos colaborativos. <https://educafinanzas.org/>
17. PHP.net. (2023). Documentación oficial de PHP. <https://www.php.net/manual/es/>
18. Ramírez, A. (2021). Manual completo de desarrollo web con PHP y MySQL. Editorial Trillas.
19. Ramírez, F. (2023). Desarrollo con FullCalendar JS: uso en sistemas de agendamiento. Blog de Ingeniería de Software. <https://blogs.devfullcalendar.com/>
20. Sánchez, A. (2022). Pruebas unitarias y de integración en PHP. Curso en línea Edutin. <https://edutin.com/>
21. Sánchez, E. (2022). Buenas prácticas en CSS3: diseño web moderno y responsivo. Revista Frontend UX. <https://revistafrontendux.com/>
22. Scrum.org. (2021). Guía Scrum: La Guía Definitiva de Scrum. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish.pdf>
23. Torres, M. (2022). Fundamentos de JavaScript moderno. WebReactiva. <https://webreactiva.com/>
24. UNAM. (2020). Ingeniería de Software: fundamentos y ciclo de vida del software. Cursos Masivos Abiertos UNAM. <https://cursosabiertos.unam.mx/>
25. Universidad de Alicante. (s.f.). Gestión de riesgos en proyectos de software. <https://rua.ua.es/>
26. Vega, J. (2022). Cómo elaborar un Product Backlog correctamente. Think Agile. <https://www.thinkagile.com.mx/>
27. Villalobos, C. (2021). Modelo FURPS: Clasificación de Requisitos en Ingeniería de Software. <https://ingenieriasystems.com/>
28. W3Schools. (2024). Guías prácticas de desarrollo web: HTML, CSS, JavaScript, PHP y MySQL. <https://www.w3schools.com/>

13. Apéndices.

PHP

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Acceso Administrador | Hospital Infantil</title>
  <link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/awesome/6.4.0/css/all.min.css">
  <link rel="stylesheet" href="../css/estilo-hospital.css">
</head>
<body>
  <div class="login-container">
    <div class="logo-container">
      
      <h1>Acceso Administrativo</h1>
    </div>
    <div class="mb-4 text-center">
      <h4>Inicio de sesión</h4>
      <form id="adminLoginForm">
        <div class="mb-3">
          <label for="adminEmail" class="form-label">Correo electrónico</label>
          <input type="email" id="adminEmail" class="form-control"
placeholder="ejemplo@correo.com" required>
        </div>
        <div class="mb-4">
          <label for="adminPassword" class="form-label">Contraseña</label>
          <input type="password" id="adminPassword" class="form-control" required>
        </div>
        <button type="submit" class="btn btn-custom w-100">Ingresar</button>
      </form>
      <div class="text-center mt-4">
        <a href="../php/forgot_password.php">¿Olvidaste tu contraseña?</a>
      </div>
    </div>
  </div>

  <!-- Bloque de script donde importas authenticateAdmin y manejas el submit -->
  <script type="module">
    import { authenticateAdmin } from '../js/storage.js';

    const form = document.getElementById('adminLoginForm');
    form.addEventListener('submit', async (e) => {
      e.preventDefault();
      const email = document.getElementById('adminEmail').value.trim();
      const password = document.getElementById('adminPassword').value;

      try {
        await authenticateAdmin(email, password);
        window.location.href =
'https://hic.hospitalinfantil.org.mx/Administrador/dashboard.php';
      } catch (err) {
        alert(err.message || 'Credenciales incorrectas');
      }
    });
  </script>
</body>
</html>

```

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Acceso Usuarios | Hospital Infantil</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <link href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css" rel="stylesheet">
  <link rel="stylesheet" href="../css/estilo-hospital.css">
  <style>
    body {
      background: linear-gradient(to right, #b695e5, #f7f0ff);
      font-family: 'Montserrat', sans-serif;
    }
    .login-container {
      max-width: 450px;
      margin: 5rem auto;
      background: white;
      padding: 2.5rem;
      border-radius: 12px;
      box-shadow: 0 0 15px rgba(0,0,0,0.1);
    }
    .logo-container { text-align: center; margin-bottom: 2rem; }
    .logo-container img { width: 100px; margin-bottom: 1rem; }
    .logo-container h1 { font-size: 1.5rem; font-weight: bold; color: #5b2c6f; }
    .btn-custom { background-color: #5b2c6f; color: white; }
    .btn-custom:hover { background-color: #4a2459; }
    a { color: #5b2c6f; font-weight: 500; }
  </style>
</head>
<body>
<main>
  <div class="login-container">
    <div class="logo-container">
      
      <h1>Acceso Usuario</h1>
    </div>

    <!-- Paso 1: Usuario + Contraseña -->
    <form id="userLoginForm">
      <div class="mb-3">
        <label for="username" class="form-label">Usuario</label>
        <input type="text" id="username" class="form-control" placeholder="Tu usuario"
required>
      </div>
      <div class="mb-4">
        <label for="userPassword" class="form-label">Contraseña</label>
        <input type="password" id="userPassword" class="form-control" required>
      </div>
      <button type="submit" class="btn btn-custom w-100">Siguiente</button>
    </form>

    <!-- Paso 2: Expediente (oculto inicialmente) -->
    <form id="expedienteForm" style="display:none">
      <div class="mb-4">
        <label for="expediente" class="form-label">Número de expediente</label>
        <input type="text" id="expediente" class="form-control" placeholder="123456"
required>
      </div>
      <button type="submit" class="btn btn-custom w-100">Ingresar</button>
    </form>

    <div class="text-center mt-4">
      <a href="../php/forgot_password.php">¿Olvidaste tu contraseña?</a>
    </div>
  </div>
</main>

<script type="module">
  const loginForm = document.getElementById('userLoginForm');
  const expedienteForm = document.getElementById('expedienteForm');

  loginForm.addEventListener('submit', async e => {
    e.preventDefault();
    const username = document.getElementById('username').value.trim();
    const password = document.getElementById('userPassword').value;
    try {
      const res = await fetch('../php/login_user.php', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ username, password })
      });
      const data = await res.json();
      if (!res.ok) throw new Error(data.message);
      // Mostrar paso 2
      loginForm.style.display = 'none';
      expedienteForm.style.display = 'block';
      alert(data.message);
    } catch (err) {
      alert(err.message || 'Error al validar credenciales');
    }
  });

  expedienteForm.addEventListener('submit', async e => {
    e.preventDefault();
    const expediente = document.getElementById('expediente').value.trim();
    try {
      const res = await fetch('../php/validate_expediente.php', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ expediente })
      });
      const data = await res.json();
      if (!res.ok) throw new Error(data.message);
      // Login completo, redirigir a área de usuario
      window.location.href = '/Usuario/calendario_tutor.html';
    } catch (err) {
      alert(err.message || 'Expediente inválido');
    }
  });
</script>
</body>
</html>

```


Comentarios finales

Si bien el proyecto al inicio tenía un nombre de calendarización de citas médicas y pagos, al ir a las reuniones se nos informó que solo manejaríamos la calendarización, por lo cual el área de pagos quedaba descartada.



Jose Raziel Trujeque Castor

para consultaexterna, informatica, comunicacion, Ingeniero, Reyes, Armando, mí, Carlos, Mauricio ▾

mar, 29 abr, 15:12 ☆ ↶

Traducir al español ×

Nos comunicamos con ustedes para comentar un tema relacionado con el nombre del proyecto.

Al inicio, el proyecto fue registrado con el título "**Proyecto de Gestión de Citas Médicas de Calendarización y Pagos**". Sin embargo, al comenzar con la recolección de requerimientos, se nos indicó por parte de la administración del hospital que únicamente trabajaríamos en la parte de **calendarización de citas**, dejando pendiente la funcionalidad de **pagos** por motivos contables.

Por esta razón, nuestros supervisores de la Unidad Académica nos han solicitado una confirmación por escrito para formalizar el cambio en el **nombre del proyecto**, con el fin de evitar posibles malentendidos al momento de su evaluación final.

Les agradeceríamos mucho si pudieran responder a este mensaje confirmando dicho cambio. Aprovechamos también para recordarles que nuestra próxima reunión será este **viernes 2 de mayo a las 12:00 PM**, donde presentaremos un nuevo adelanto del proyecto.

Quedamos atentos a su respuesta.
Muchas gracias de antemano.

Atentamente,
Raziel Trujeque



Mtra. Cris García Rodríguez <consultaexterna@hospitalinfantil.org>

para Jose, informatica, comunicacion, Ingeniero, Reyes, Armando, mí, Carlos, Mauricio ▾

mar, 29 abr, 16:46 ☆ ↶ ⋮

Traducir al español ×

Buen día, confirmo la próxima reunión para seguimiento del proyecto, ustedes indiquenos si les es posible asistir al hospital o prefieren que lo realicemos por zoom.

Así mismo confirmo, debido a que no fue posible cambiar formalmente el nombre del proyecto y por motivos internos la gestión de **pagos** se omitirá del proyecto, siendo solo la calendarización de citas lo que estarán desarrollando.



CONSULTE NUESTRO AVISO DE PRIVACIDAD:

<http://hospitalinfantil.org/aviso-de-privacidad/>

